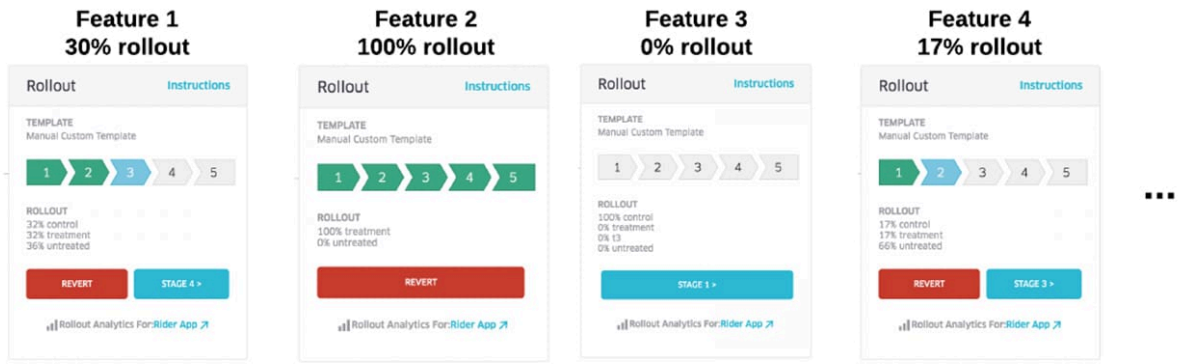
Building Automated Feature Rollouts on Robust Regression Analysis

■ uber.com/en-PL/blog/autonomous-rollouts-regression-analysis

July 17, 2018



At Uber, engineering teams launch features for our various mobile apps simultaneously across cities around the world, over multiple time zones, and in products on a daily basis. To ensure that these features land safely, we leverage our experimentation platform, a tool that lets us monitor how each new feature affects our software and systems through a staged rollout process.

In addition to running a thorough test on our code before deployment, we want to understand whether a new feature might inadvertently interact with existing features on our apps in an unexpected way. Traditionally, fixing these issues required developers to dedicate a significant amount of time investigating the source of ongoing regressions, or bugs, and mapping them back to the features that caused them. This process usually slows down feature development and can negatively impact user experience.

Therefore, we built a new system letting us quickly and easily identify which feature might be causing an issue. Along with its user-friendly analytics dashboard, the system enables scheduled autonomous feature rollouts, since we can detect and rollback features that exhibit any problems. As engineers can more quickly detect the cause of a regression, we can lessen the impact on customers by fixing any issues before they become widespread.

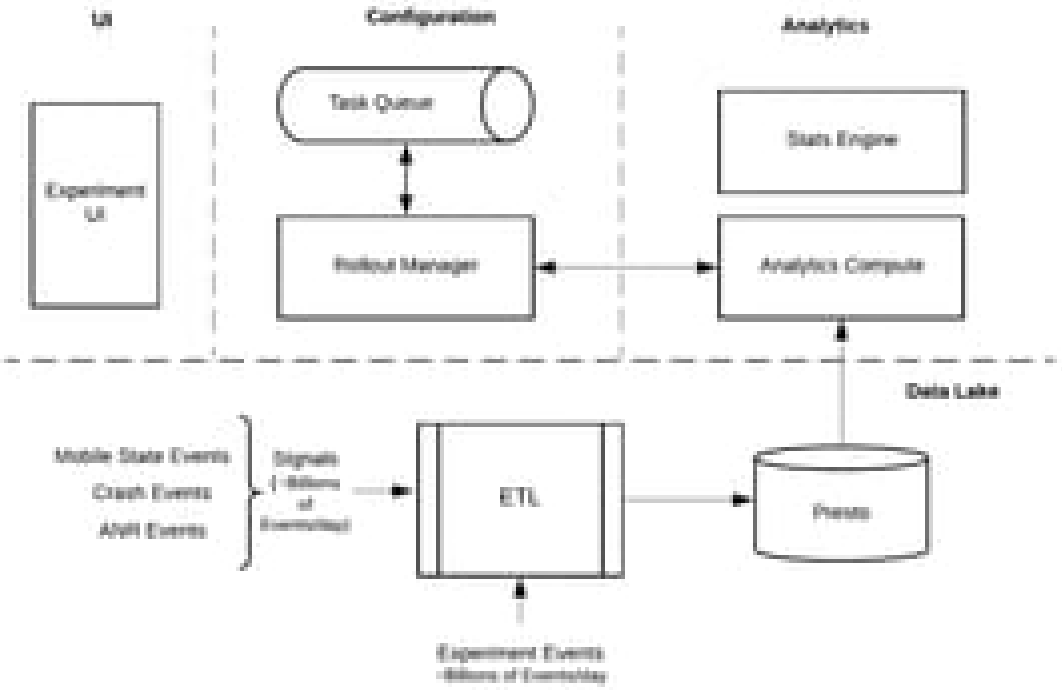


Figure 1: The Auto Rollouts system consists of a configuration layer written in Go, an analytics engine written in Java and Scala, a user interface layer written in ReactJs, and a data lake, where the ETLs are run, that leverages Hadoop and Presto.

Enhancing attribution and regression detection

Our existing staged rollout procedure uses a methodology borrowed from scientific experimentation: as we roll a new feature out to customers, the set of end users who receive it are considered the *treatment* group, while the remaining end users are the *control* group. We collect events from the treatment and control groups, correlating them with different health metrics to check for regressions. If we detect a regression, we can quickly roll back the deployment and fix the problem.

Our engineering colleagues, Eva Feng and Zhenyu Zhao, detailed this staged rollout procedure last year in their article, *Building an Intelligent Experimentation Platform with Uber Engineering*.

To enable our Auto Rollouts system, we built up the existing framework by enhancing its monitoring capabilities and adding an analytics dashboard, which lets developers easily look into the cause of a regression. Now, if we detect a regression, we can attribute it to the feature that caused it, the owner who created the feature, and the part of the app where the regression was introduced.

During a deployment, the system looks for app state events, crash events, and application-not-responding (ANR) events among the metrics that continually pour in over our production platform. The system correlates those metrics with the events from the feature being deployed, then passes the results through a stats processor and decision engine to determine whether

there is a regression. The output of the decision engine determines which action the Auto Rollout system takes, either reverting in case of a regression or else proceeding with the feature deployment.

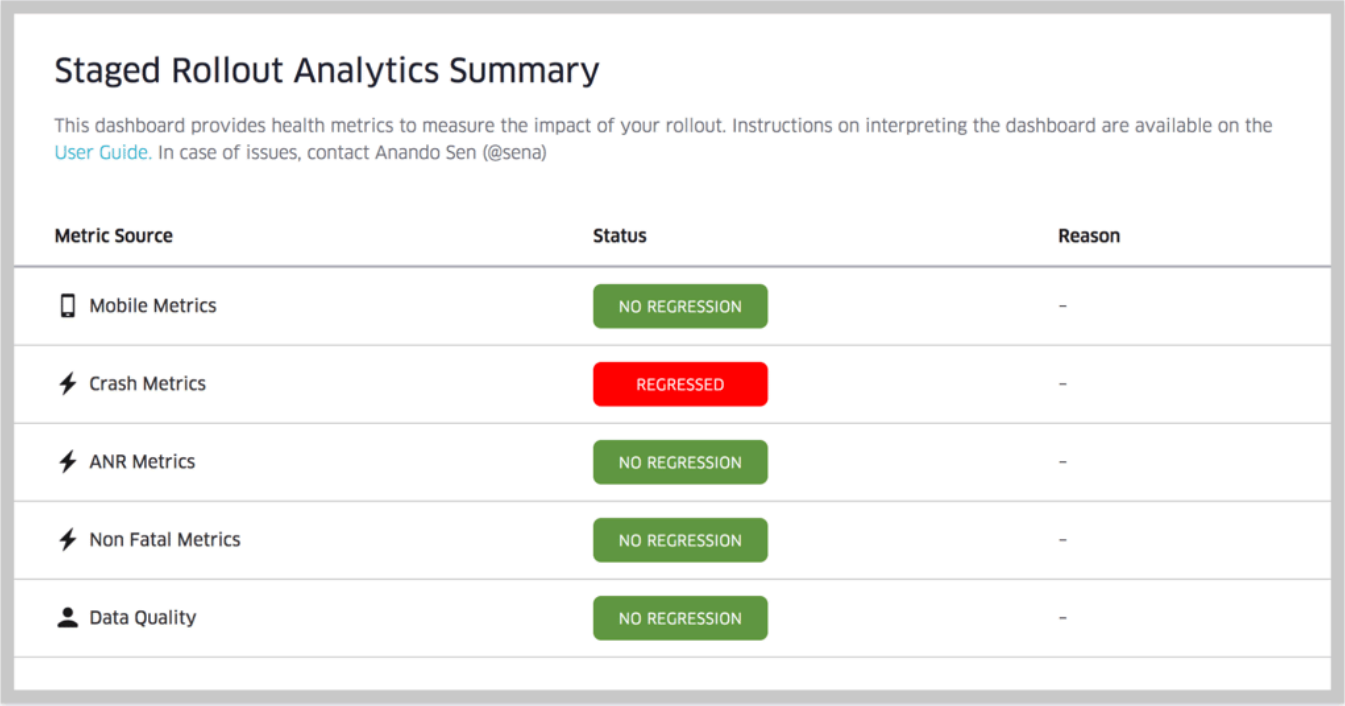


Figure 2: To make regressions more visible during a rollout, we built an analytics dashboard that provides an overview of the impact of a feature across different metrics.

Our enhanced monitoring lets us dig further into these metrics to identify events that are undergoing a regression. In Figure 3, below, the new feature caused a crash event, identified as *crash_f6ddab…* In its Crash Metrics section, our dashboard enables us to determine that this type of crash occurs 10 percent more frequently in the treatment flow than in the control flow.

Crash Metrics

REGRESSED

Shows the percentage of users in treatment and control that are correlated with the crash event

USER LEVEL

Metric	Status	Rel. Change	Rel. Confidence Interval
crash_d208e5a9d05e5bff2eb6958c632da344	No Regression	-10%	[-310%, 110%]
crash_9c334724826751fbdca28ea7f9f3e149	No Regression	-10%	[-272%, 71.8%]
crash_7fdf45f055eee4f815c1f21dd55883ea	No Regression	0%	[-21.5%, 221%]
crash_f580958a46a0aaff8dcbd8138e518ad5	No Regression	-10%	[-288%, 88.2%]
crash_e291c0149fb955f87ca3ef1eebbab601	No Regression	0%	[-110%, 310%]
crash_69ffc43b2b3810a616935602c7166401	No Regression	0%	[-166%, 605%]
crash_f6ddab85833eabd027803cf911fb0341	Regressed	10%	[5.90%, 194%]

Figure 3: The Crash Metrics display of the analytics dashboard shows which crash events are being regressed.

The Mobile Metrics section of the analytics dashboard highlights the impact of the new feature on the different parts of the app. The dashboard in Figure 4, below, indicates that the feature had no significant impact on the confirmation flow, payment flow, or support flow of the app:

Mobile Metrics

NO REGRESSION

Shows the percentage of users (or sessions) in treatment and control that contain the mobile analytics event

USER LEVEL SESSION LEVEL

Metric	Status	Rel. Change	Rel. Confidence Interval
payment_feature_nopayment_plusone_success	No Regression	+0.691%	[-190%, 191%]
support_form_submit	No Regression	-0.368%	[-31.7%, 31.0%]
cancellation_action_sheet_accept	No Regression	+0.379%	[-348%, 349%]
confirmation	No Regression	+0.0207%	[-0.341%, 0.382%]

Figure 4: The Mobile Metrics panel shows if any of the flows were affected by the new feature.

Our enhanced staged rollouts system lets developers assess the impact of the new feature being deployed, and any regressions detected in Uber's platform can be attributed back to that feature.

Science behind the analytics

Underlying the Auto Rollouts system, our experimentation platform computes each metric it uses by correlating it with experiment events. To enable automated rollouts, we improved regression detection so the framework could more accurately initiate a rollback.

The regression detection algorithm run on each metric is powered by a sequential likelihood ratio test. We look at the following events to compute the metrics analyzed by this system:

- **Mobile Events: The mobile app emits a checkpoint event in the app's sections. These events include 'Dispatch Request', 'Signed In', and 'Request Trip', among others.**
- **Crash Events: These events are emitted when the app crashes.**
- **Application Not Responding (ANR) Events: These events are emitted when the application is stuck and will not respond to user interaction.**

Personalize metrics parameters

To determine a metric's regression, we construct the hypothesis test as:

H0: |Signal in Treatment – Signal in Control| <= Threshold
H1: |Signal in Treatment – Signal in Control| > Threshold

We construct an 'always valid' (1-alpha) confidence interval around the difference between the treatment and control groups using sequential testing,

P(|Signal in Treatment – Signal in Control| > Threshold | H0) < alpha,

where alpha = Type I error. (In statistics, a type I error means a false positive result.)

We keep monitoring until the sample size reaches our expected sample size, such that

P(|Signal in Treatment – Signal in Control| < Threshold | H1) < beta,

where beta = Type II error. (In statistics, a type II error means a false negative result.)

If no regression is detected, then the rollout is considered safe. This statistical test is constructed using a sequential testing algorithm, which enables continuous monitoring without inflating the Type I error. We monitor different metrics for Uber's rider, driver, and Uber Eats apps.

Minimum Detectable Effect (MDE) indicates the smallest change we can detect in treatment relative to the control experience. The metrics we use differ in their noise level and detectability, so we use different MDE thresholds for each. If we use an MDE threshold X, then in order to be 90 percent confident that there is no regression > X, we need to predict the number of users we need in the treatment and control flows. The graph below indicates, for a metric with mean around 0.8 to achieve an MDE threshold of 3 percent, we would need at least ~18,000 users in the treatment and control flows (combined) before we can call the rollout as being safe with 90 percent confidence.

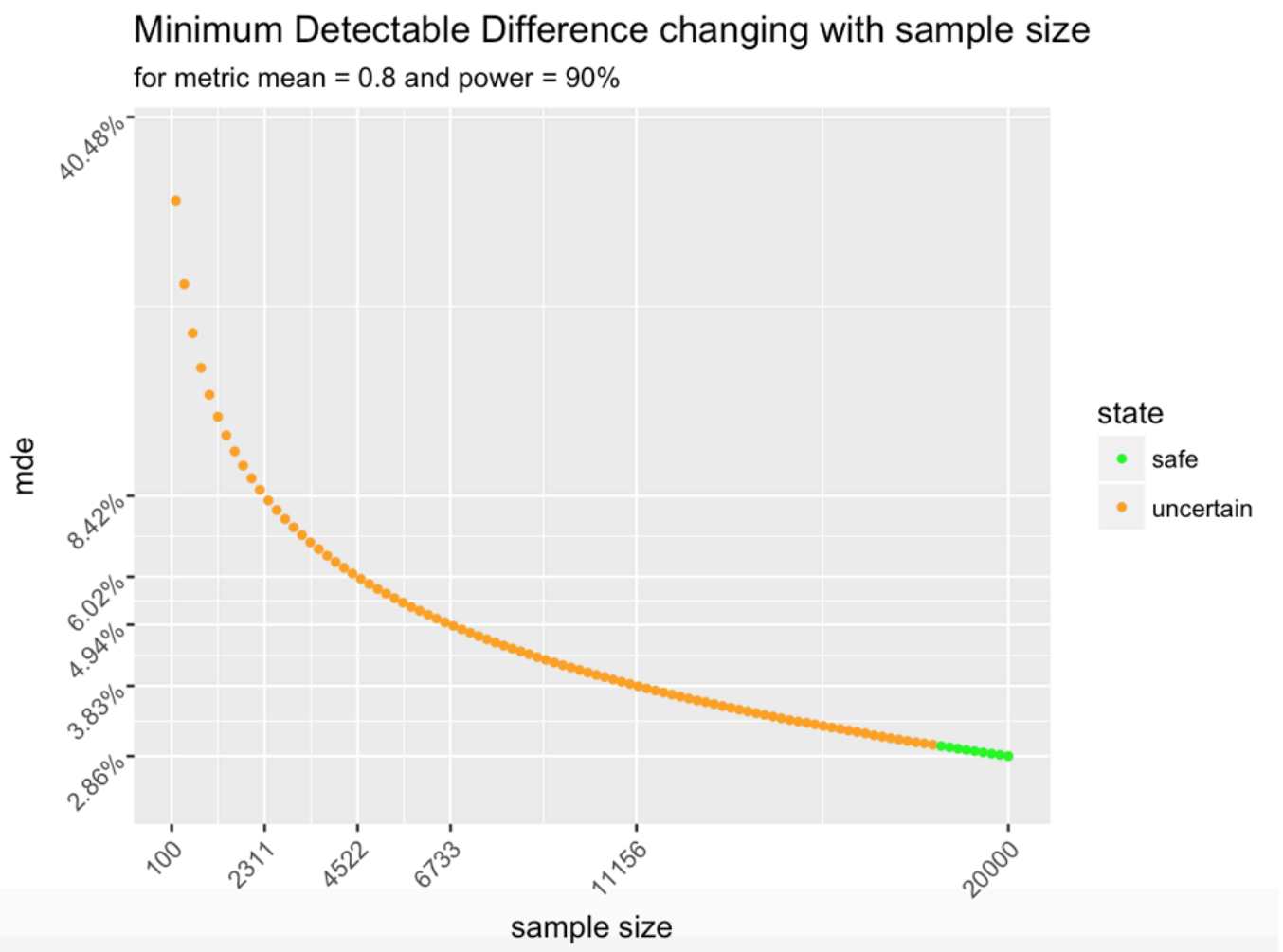


Figure 5: This graph shows how the MDE decreases as we increase the sample size.

Scheduling feature rollouts

Along with making it easier for engineers to identify problems with a new feature, our improved regression monitoring also lets us automate feature deployment, since the system can rollback a deployment if something goes wrong. Our Auto Rollouts system lessens developer fatigue, since users do not have to manually activate and monitor each step of the rollout, and makes the whole process run faster.

The Auto Rollout system consists of two main components: the Rollout UI, which lets users set rollout parameters, and the Rollout Configuration processor, managing the progress of each feature rollout through its lifecycle.

Rollout UI

The Auto Rollout system lets users set a few configuration parameters through a simple user interface. They can pick the treatment group for rollout, the start date, and the number of days over which to complete the rollout.

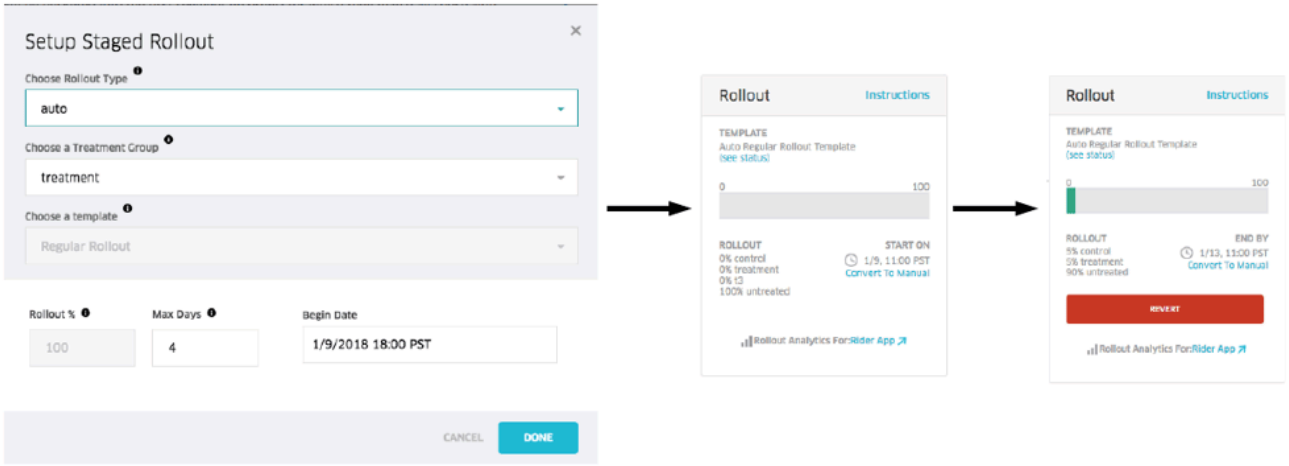


Figure 6: The UI lets developers choose to automate a rollout, and set its timing.

Rollout Configuration

Every rollout has a lifecycle, starting with initial setup and ending in completion. The Rollout Configuration processor manages the lifecycle by moving the rollout through its stages, monitoring the analytics in the background, and updating users at each stage. This hands-free approach enables users to rollout features without having to monitor them over multiple days. Engineers can immediately start their rollouts or schedule them for a future date and time.

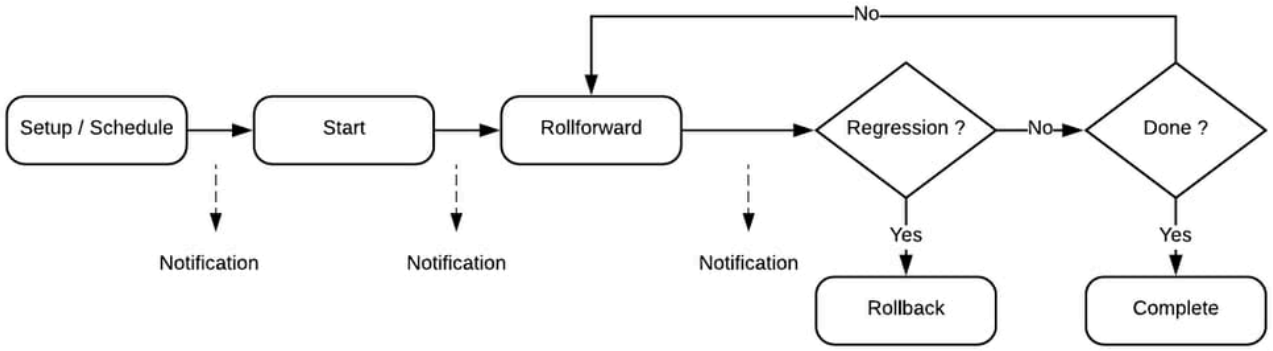


Figure 7: The Rollout Configuration processor takes a new feature through each stage of the life cycle.

Successful hands-off deployment

Since launch, this system has performed thousands of rollouts autonomously for features across Uber's apps. The addition of the Auto Rollouts system has resulted in a number of advantages for Uber's engineering workflow:

- **Attribution**: A regression can be caught and attributed to a particular feature or owner.
- **Greater stability**: Engineers can set up the system to rollout the feature any time, night or day, and be assured that it will be monitored against multiple metrics for regressions.
- **Increased speed of deployment**: The system's automation and ability to catch regressions doubled our feature rollout speed.

If you are interested in building monitored, automated, and safe feature deployment solutions, consider joining our team!

Subscribe to our newsletter to keep up with the latest innovations from Uber Engineering.

Tim Knapik is an engineer on Uber's Experimentation Platform team.

Mandie Liu is a data scientist on Uber's Experimentation Platform team.

David Schnurr is an engineer on Uber's Movement team.

Sisil Mehta is a senior software engineer on the Uber Freight team.

Anando Sen is a senior product manager on Uber's Developer Platform team.

Posted by Tim Knapik, Mandie Liu, David Schnurr, Sisil Mehta, Anando Sen

Category:
Engineering